Data Analytics Project: IGN Game Reviews Analysis

Dataset Used: ign.csv

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyze game review scores, platform popularity, and genre distribution over time using the `ign.csv` dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (🟢 Easy)

Description:
This scenario focuses on loading and understanding the dataset using Pandas.
Basic exploration is done using head(), tail(), and shape functions.
Unnecessary columns are removed to clean the dataset.
Missing values are identified and handled using mean and mode.
Finally, data types are corrected for proper analysis.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
4.	Check for missing values in `score`, `genre`, and `platform`.
5.	Handle missing values: Fill the numeric column `score` with the mean, and the categorical column `genre` with the mode.
6.	Ensure correct data types: Convert `score` → float and `release_year`, `release_month`, `release_day` → integer.

Conclusion:
The dataset was successfully cleaned and prepared for analysis.
Missing values were handled to avoid errors in further steps.
Unnecessary columns were removed to improve efficiency.

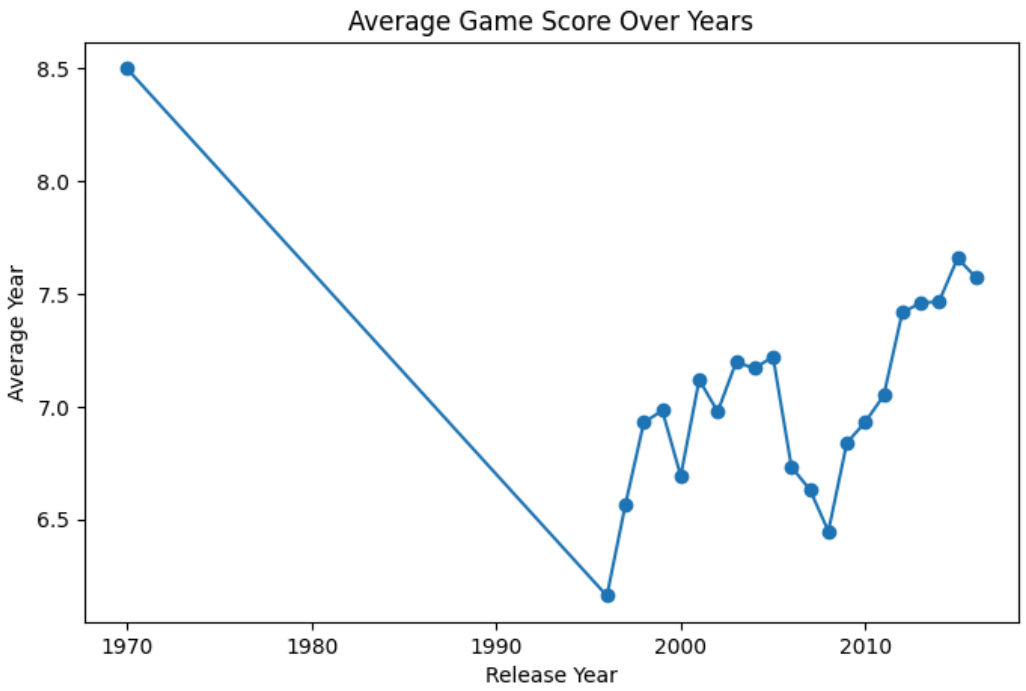
Scenario 2: Line Graph (Score Trend) + Save (🟢 Easy)

Description:
This scenario analyzes how game scores change over time.
Data is grouped by release_year to calculate average scores.
NumPy arrays are used for efficient computation.
A line graph is plotted to visualize score trends.
The graph is saved for reporting purposes.

Task	Description
1.	Group data by `release_year`.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with `release_year` on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: `plt.savefig("avg_score_trend.png")`.

Graph :

This line graph shows the trend of average game scores over the years.
The X-axis represents release years, while the Y-axis shows average scores.
It helps in understanding how game quality has changed over time.
The line pattern shows whether scores are increasing or decreasing.
This graph helps identify long-term performance trends in games.



Conclusion:

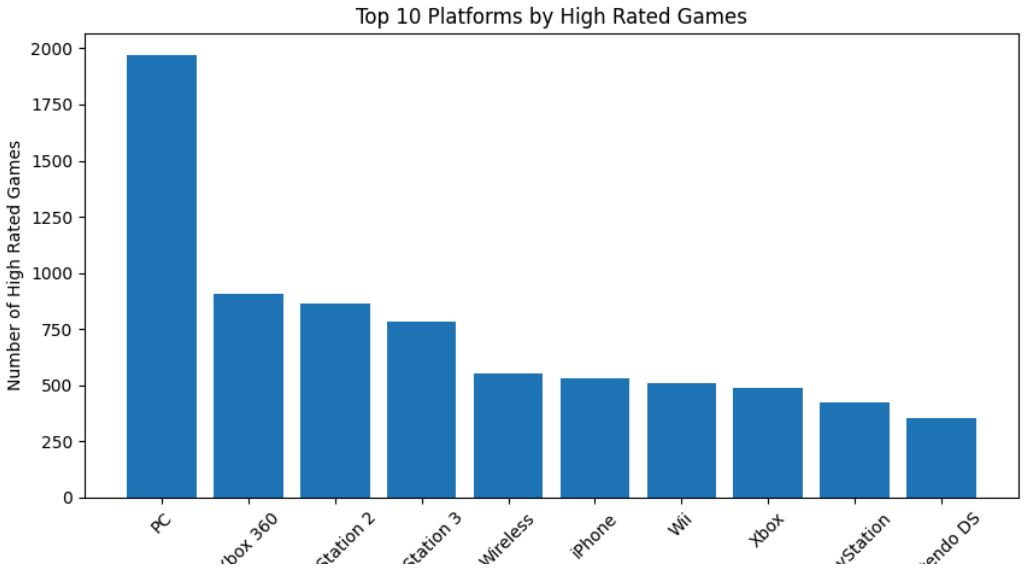
The line graph clearly shows score trends over the years.
We can identify increases or decreases in average scores.
Grouping helped simplify yearly analysis.
Visualization made patterns easy to understand.
Saved graph can be used for reports and presentations.

Scenario 3: Filtering + Bar Chart + Save (🟡 Medium)

Description:
This scenario focuses on identifying high-rated games.
Games with score greater than 7 are filtered.
The number of such games is counted per platform.
Top 10 platforms are selected for analysis.
A bar chart is plotted and saved for visualization.

Task	Description
1.	Filter the dataset to include only games where `score > 7`.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with `platform` on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: `plt.savefig("top_platforms_bar.png")`.

Graph:
This bar chart represents the top 10 platforms based on high-rated games.
The X-axis shows platforms and the Y-axis shows the number of games.
It helps compare which platforms have the most successful games.
Higher bars indicate better-performing platforms.
This graph highlights platform popularity in the dataset.



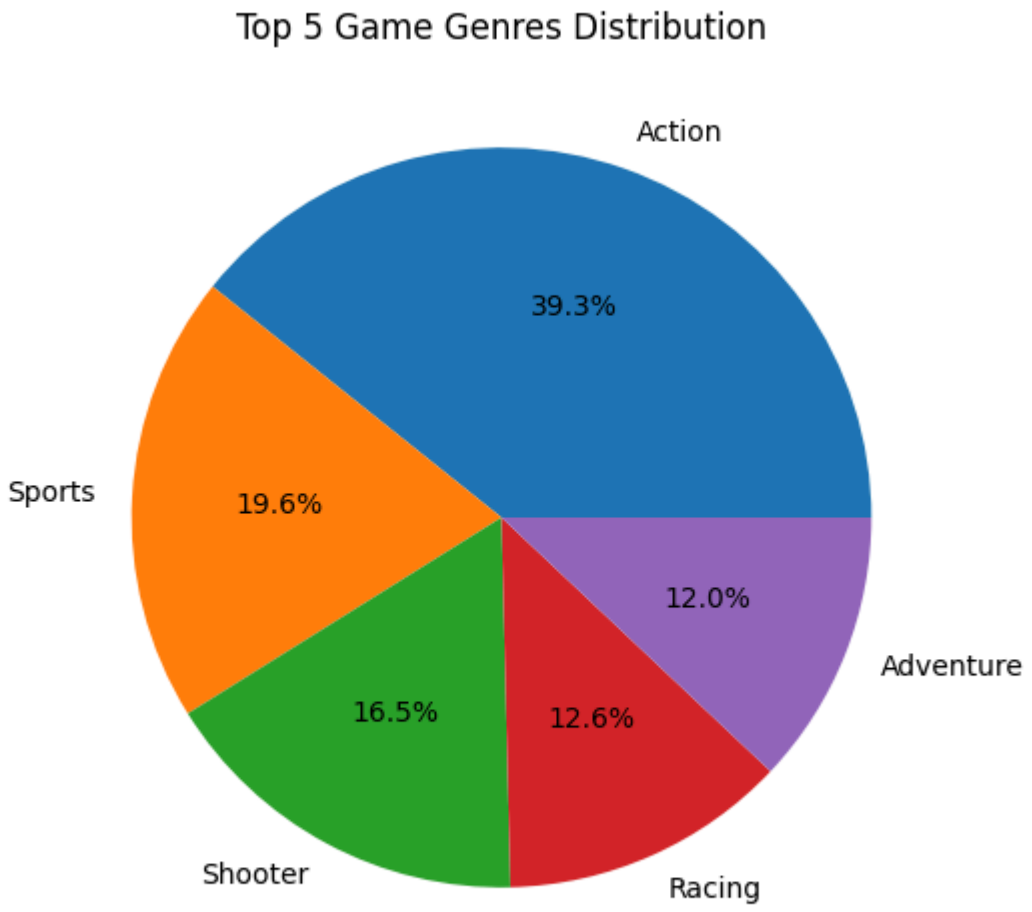
Conclusion:
Popular platforms with high-rated games were identified.
Bar chart helped compare platforms easily.
Filtering improved focus on quality games.
Top platforms showed dominance in game ratings.
This helps understand platform performance.

Scenario 4: Aggregation + Pie Chart + Save (🟡 Medium)

Description:
This scenario analyzes the distribution of game genres.
Total games per genre are counted using Pandas.
Top 5 genres are selected for clarity.
A pie chart is plotted to show proportions.
Percentage labels improve readability of the chart.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

Graph:
This pie chart shows the distribution of the top 5 game genres.
Each slice represents the percentage of games in a genre.
It helps visualize which genres are most popular.
Larger slices indicate higher dominance of that genre.
This graph gives a clear view of user preferences.



Conclusion:
The most popular game genres were identified.
Pie chart clearly showed genre distribution.
Top genres dominated the dataset significantly.
Visualization made proportions easy to interpret.
This helps understand player preferences.

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

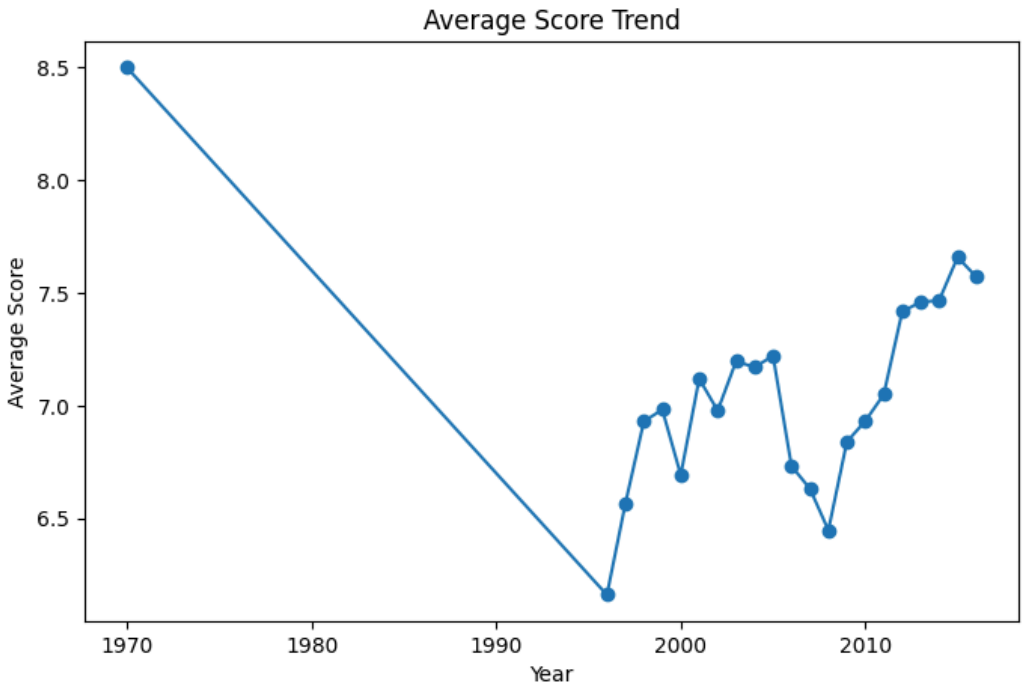
Description:
This scenario involves advanced data analysis and feature engineering.
A new column score_category is created based on score ranges.
editors_choice is converted into numeric format.
Multiple graphs like line chart, stacked bar, and histogram are plotted.
NumPy is used to calculate yearly score growth.

Task	Description
1.	Create a new column `score_category`: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the `editors_choice` column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's `np.diff()` to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per `release_year`.
5.	Plot a **Stacked Bar Chart** showing the count of `score_category` per `release_year`.
6.	Plot a **Histogram** showing the distribution of `score`.
7.	Save the graphs: `plt.savefig("score_trend.png")`, `plt.savefig("score_category_stacked.png")`, `plt.savefig("score_distribution.png")`.
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if `editors_choice` correlates with high scores.

Graph:

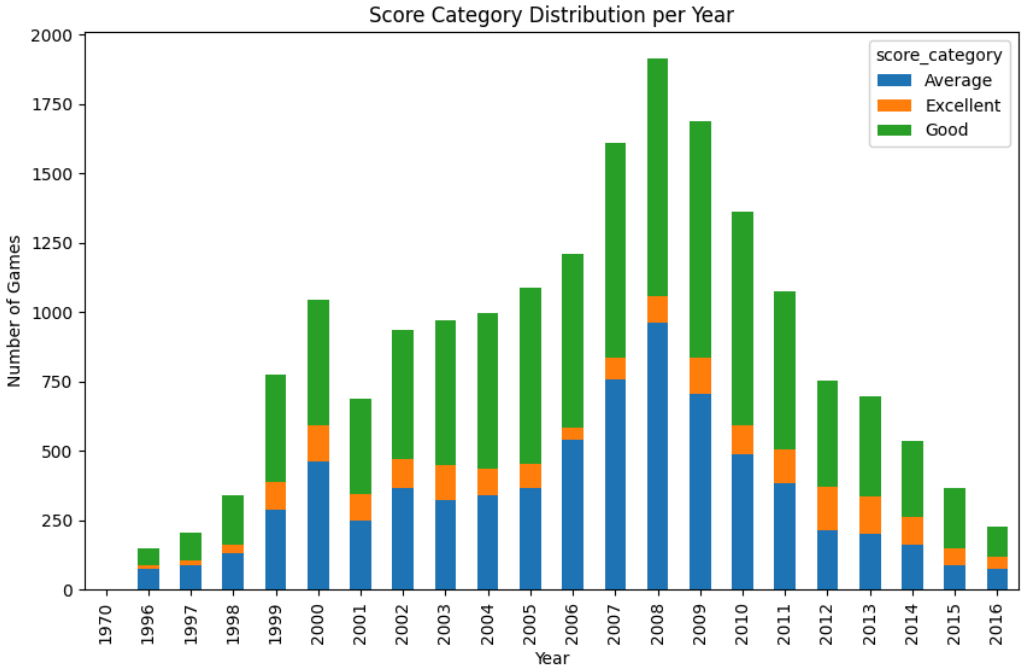
📈 Line Graph

This graph shows the trend of average game scores over time.
It helps understand how game quality changes year by year.
The pattern shows whether scores are improving or declining.
It provides a clear time-based performance analysis.
It is useful for identifying long-term trends.



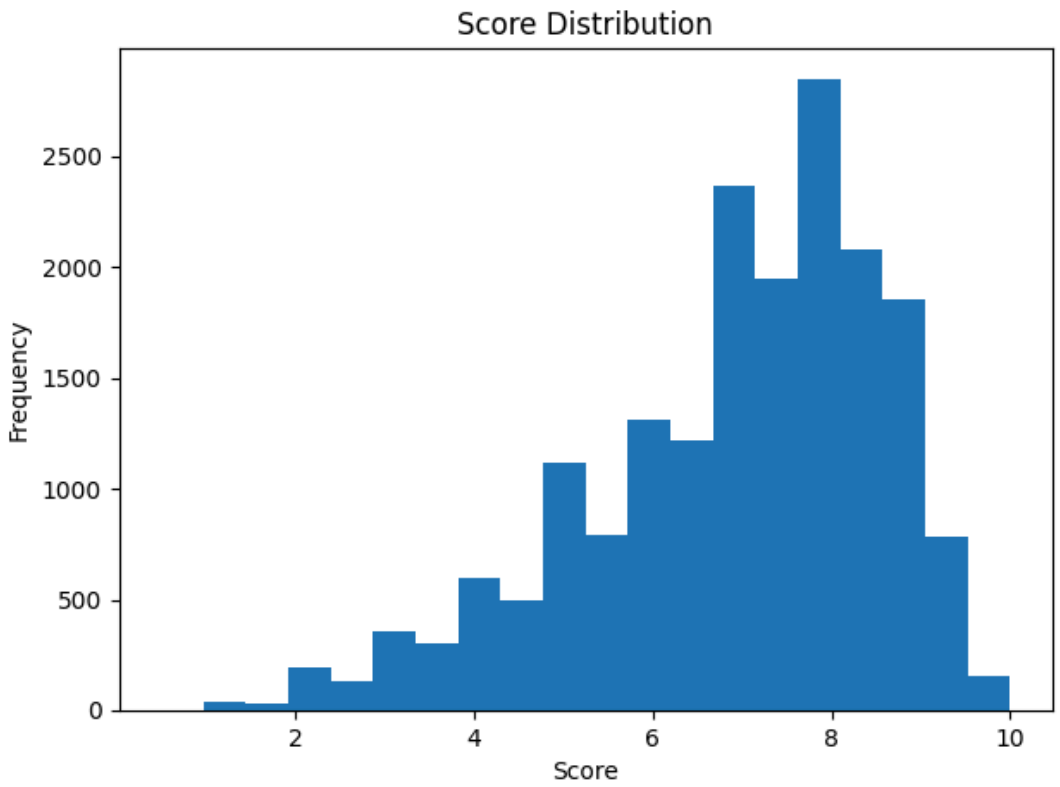
📊 Stacked Bar Chart

This graph shows score categories (Excellent, Good, Average) per year.
It helps compare how game quality distribution changes over time.
Each color represents a different score category.
It highlights yearly quality patterns clearly.
It makes comparison between years easy.



📉 Histogram

This graph shows the distribution of game scores.
It helps understand how frequently different scores appear.
Most common score ranges are clearly visible.
It shows whether data is skewed or balanced.
It helps analyze overall rating patterns.



Conclusion:
Advanced analysis revealed deeper insights into the dataset.
Score categories helped classify game quality.
Graphs showed trends, distributions, and comparisons.
Correlation analysis gave insight into editor choices.
This scenario combined all skills for complete analysis.

Final Conclusion:

The project successfully analyzed game reviews using Pandas, NumPy, and Matplotlib.
Data was cleaned, processed, and visualized effectively.
Different scenarios helped understand trends, platforms, and genres.
Visualizations made insights clear and easy to interpret.
Overall, the project provided a complete data analysis workflow.

What you have learned

Learned how to clean and preprocess real-world data.
Understood grouping, filtering, and aggregation in Pandas.
Used NumPy for numerical analysis like growth calculation.
Created multiple types of visualizations using Matplotlib.
Gained practical experience in end-to-end data analysis project.